Assignment MERN Stack Authentication & CRUD with MySQL

Name: Naveen Kumar R
USN:4MH22CA027
Sem: 8th Sem
Collage : Maharaja Institute of technology, Mysuru

Mentor:

Jacob Dennis

Institution:

CampusPe

Tech Stack: MySQL, Express.js, React.js, Node.js

Project Development Report: Full Stack Management System

1. Project Overview

This report documents the development and implementation of a full-stack web application designed for item management. The system features a robust backend for data persistence and authentication, paired with a modern, responsive frontend dashboard.

2. Features Implemented

2.1 Core Authentication

- **User Registration & Login:** Secure onboarding for new users.
- **Security:** Implementation of password hashing (Bcrypt) and JWT (JSON Web Token) for stateless session management.
- **Authorization:** Protected routes ensuring only authenticated users can access the dashboard.
- **Logout:** Securely terminating user sessions.

2.2 Dashboard & CRUD Operations

- **Data Management:** Full Create, Read, Update, and Delete (CRUD) capabilities for items.
- **Status Selection:** Ability to categorize and track the lifecycle of items.
- **Real-time Updates:** Interface updates immediately upon data modification.

2.3 Advanced & Bonus Features

- **Global Search:** Dynamic filtering of items via a search bar.
- **Dark Mode:** A user-friendly UI toggle for better accessibility and user preference.
- **Data Portability:** Implementation of "Export to CSV" for external reporting.
- **Analytics:** A dedicated statistics display to visualize data trends and totals.

3. Technology Stack

- **Frontend:** React.js, Axios, React Router.
- **Backend:** Node.js, Express.js.
- **Database:** MySQL (via mysql2).
- **Security:** Bcrypt.js, JSON Web Tokens.
- **Environment Management:** Dotenv, Nodemon.

4. Installation & Setup

4.1 Backend Configuration

1. Navigate to the backend directory: cd backend

2. Install necessary dependencies:
   npm install express mysql2 cors bcryptjs jsonwebtoken dotenv nodemon

3. Initialize the server:
   npx nodemon
   server.js

   Note: The backend runs on http://localhost:5000.

4.2 Frontend Configuration

1. Open a new terminal and navigate to the frontend directory: cd frontend

2. Install necessary dependencies:
   npm install axios react-router-dom

3. Start the React development server: npm start

   Note: The frontend runs on http://localhost:3000.

5. API Documentation

5.1 Authentication Endpoints

Method	Endpoint	Description
POST	/api/auth/register	Register a new user account
POST	/api/auth/login	Authenticate user and receive JWT
GET	/api/auth/me	Retrieve profile of the currently logged-in user

5.2 Item Management Endpoints

Method	Endpoint	Description
GET	/api/items	Retrieve a list of all items
GET	/api/items/:id	Retrieve details for a specific item
POST	/api/items	Create a new item entry
PUT	/api/items/:id	Update an existing item
DELETE	/api/items/:id	Remove an item from the database
GET	/api/items/stats/all	Fetch calculated dashboard statistics

6. Execution Instructions

To run the project locally, follow these steps in order:

1. **Start Backend:**
   cd backend && npx nodemon server.js

2. **Start Frontend:**
   cd frontend && npm start

3. **Access Application:**
   Open your browser and navigate to http://localhost:3000.

7. Conclusion

The project successfully meets all functional requirements, providing a secure and efficient environment for item management. The inclusion of bonus features like CSV export and dark mode significantly enhances the user experience and utility of the software.

Screenshots

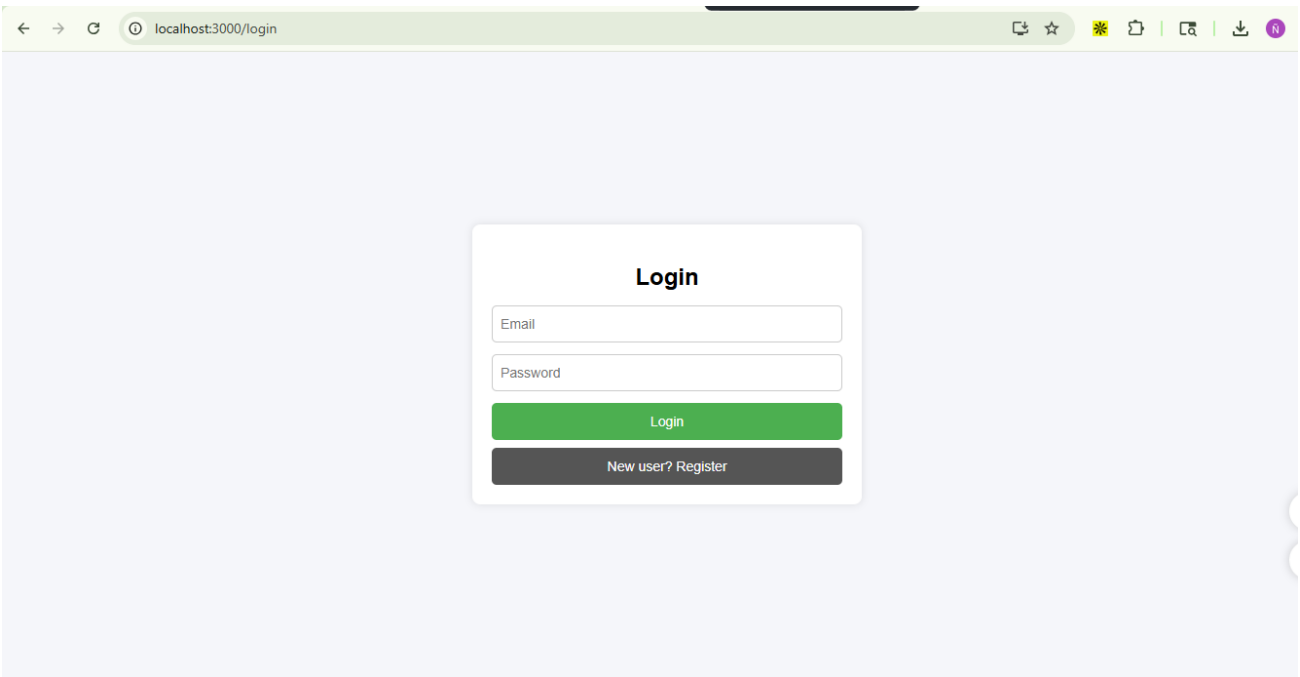

Fig 1.1 Login Page

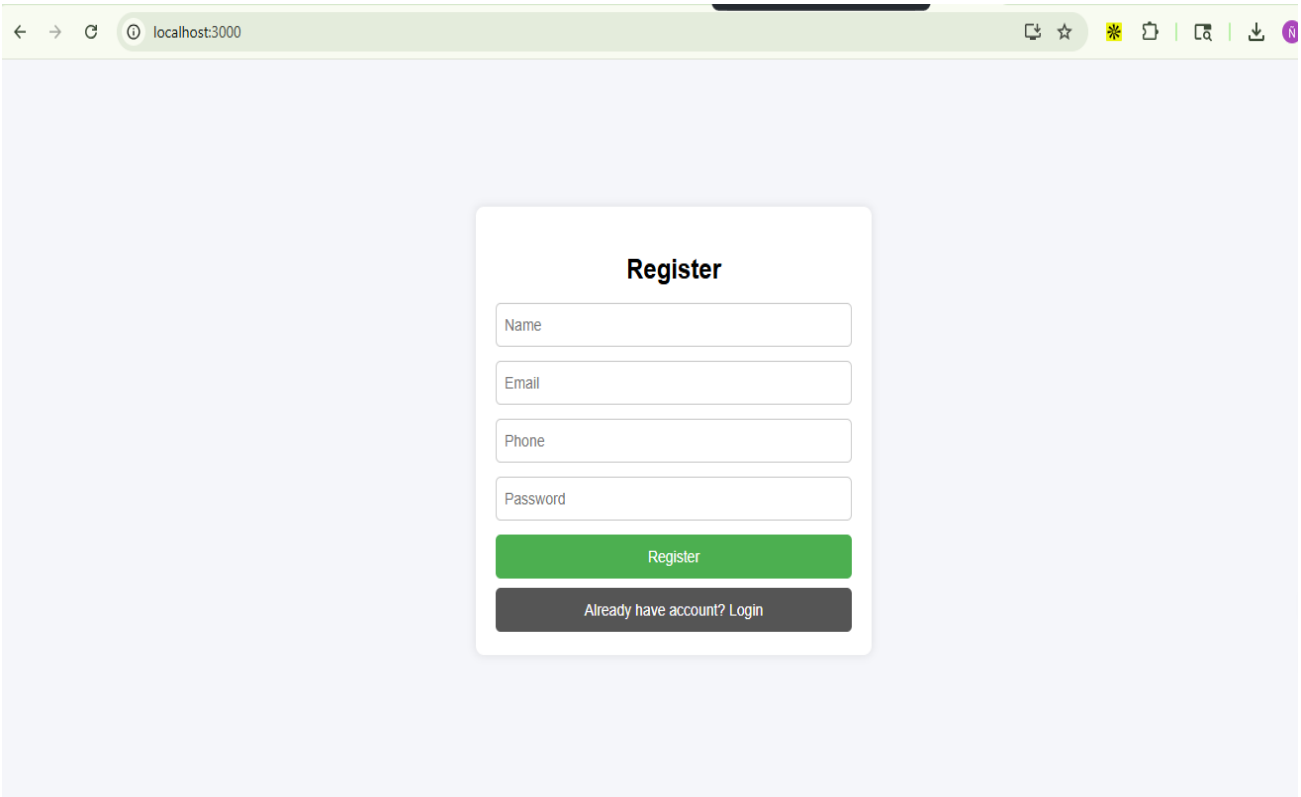

Fig 1.2 Register Page

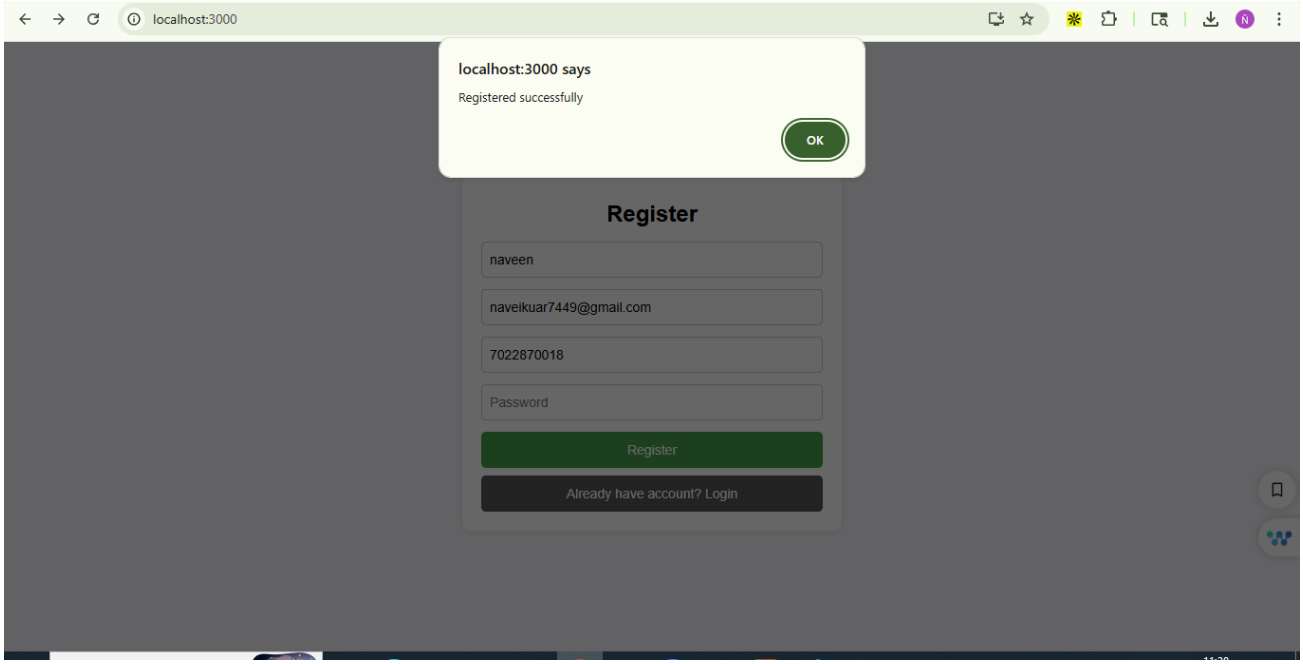

Fig 1.3 After Registration Success

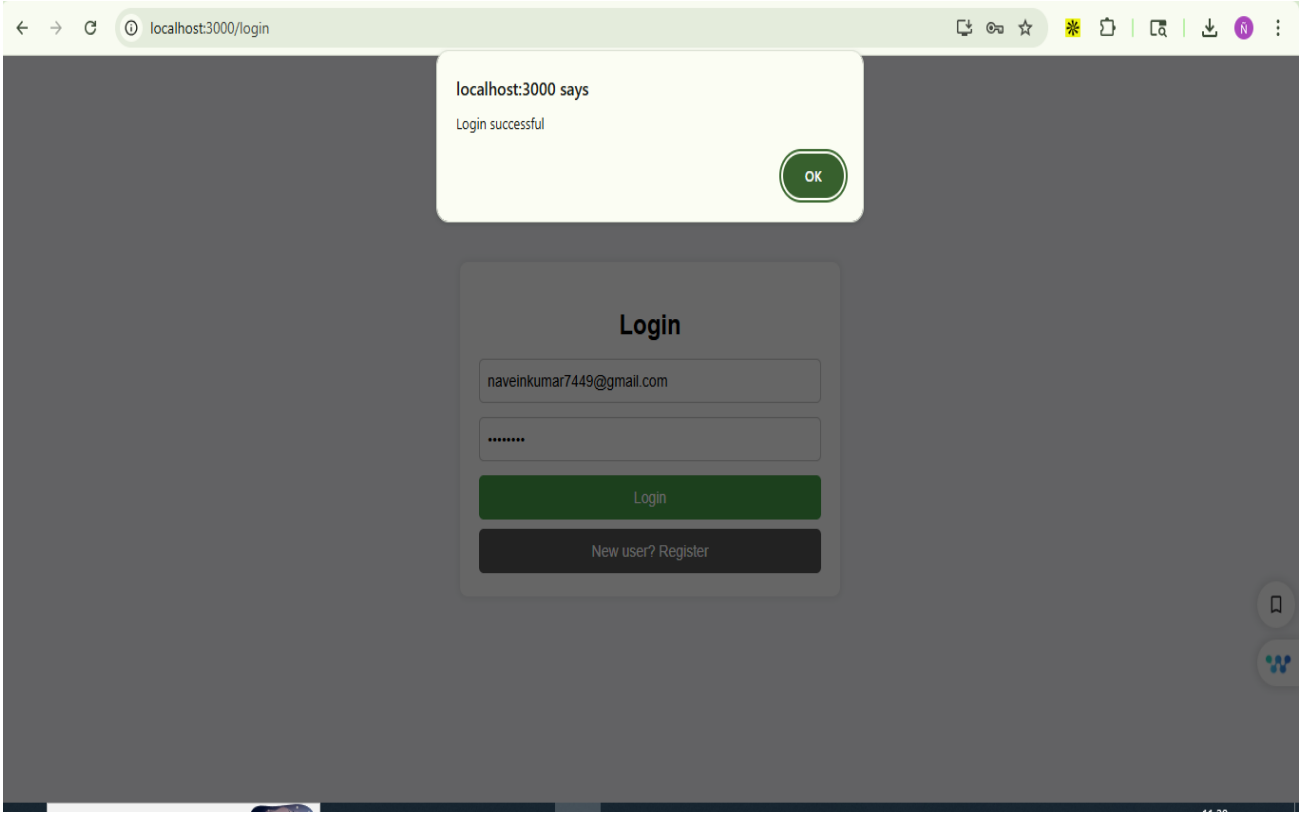

Fig 1.4 After Login Successful

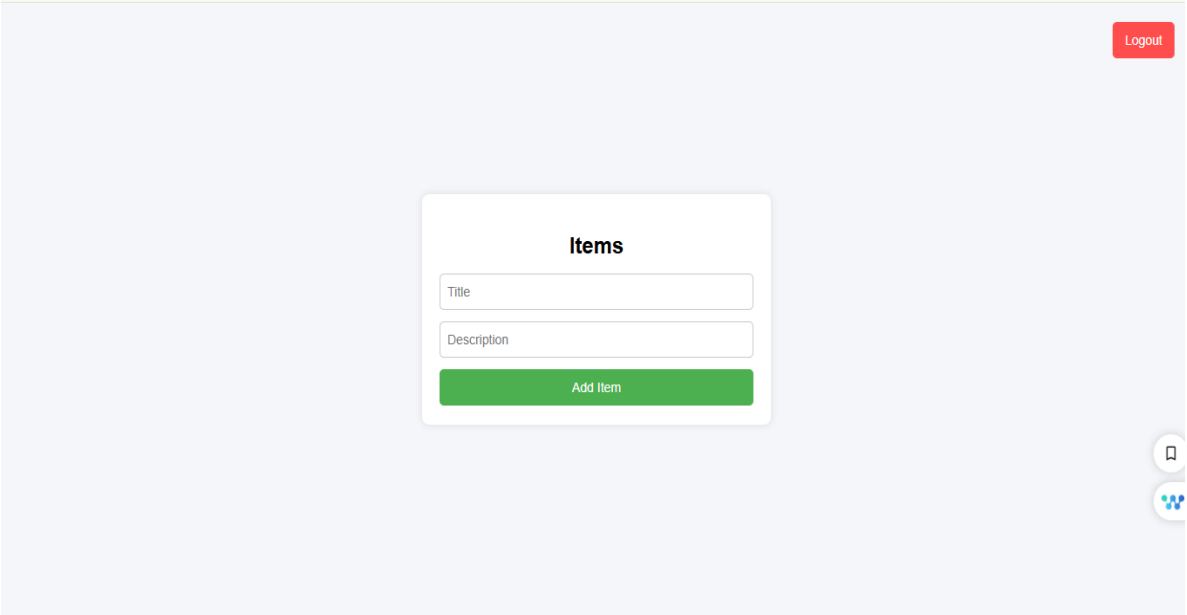

Fig 1.5 Dashboard before adding items

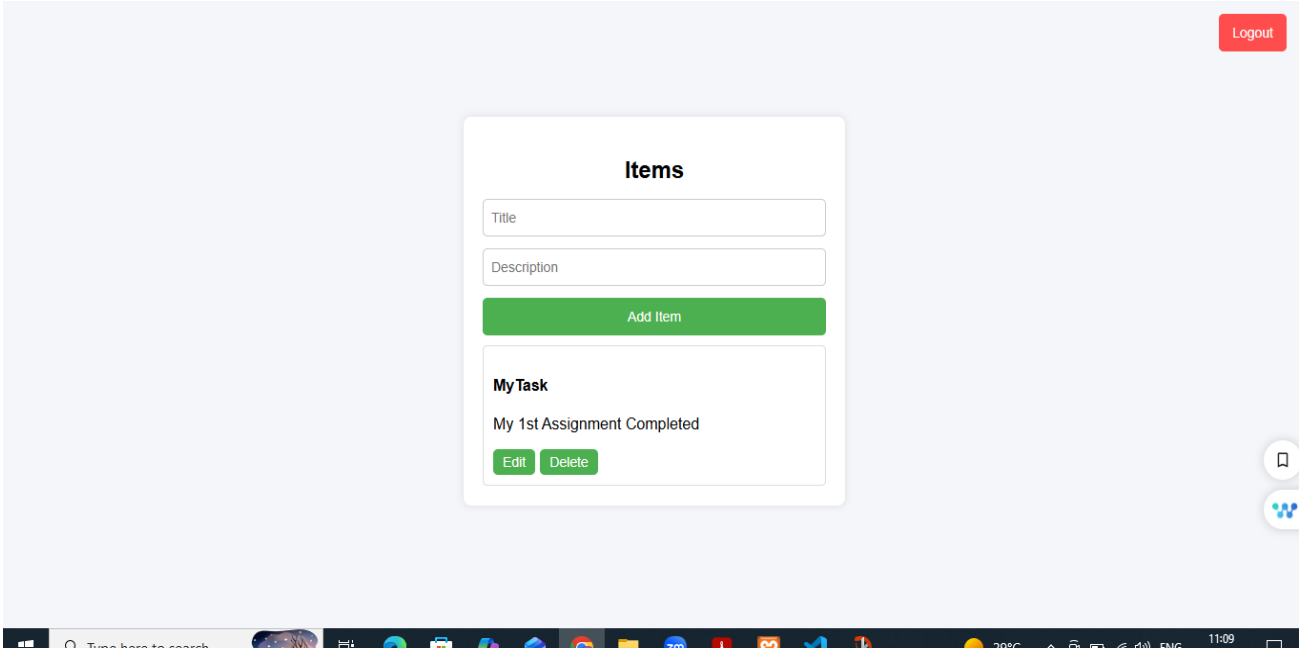

Fig 1.6 Dashboard After adding items

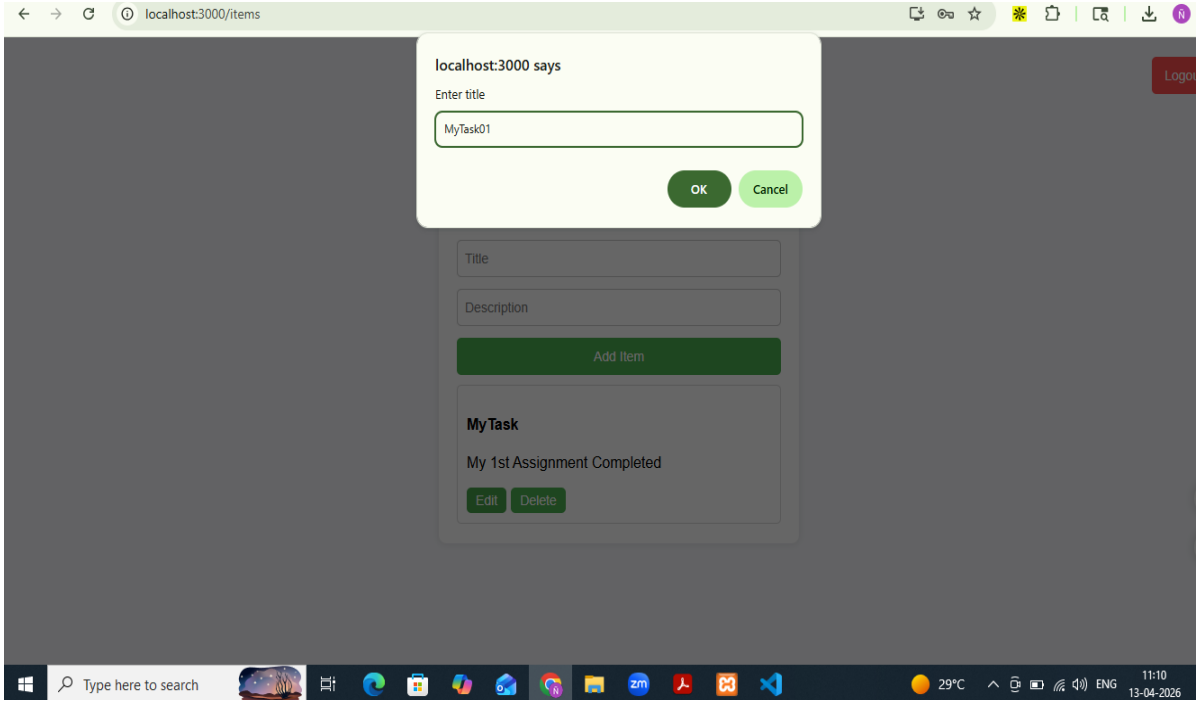

Fig 1.7 Editind Title

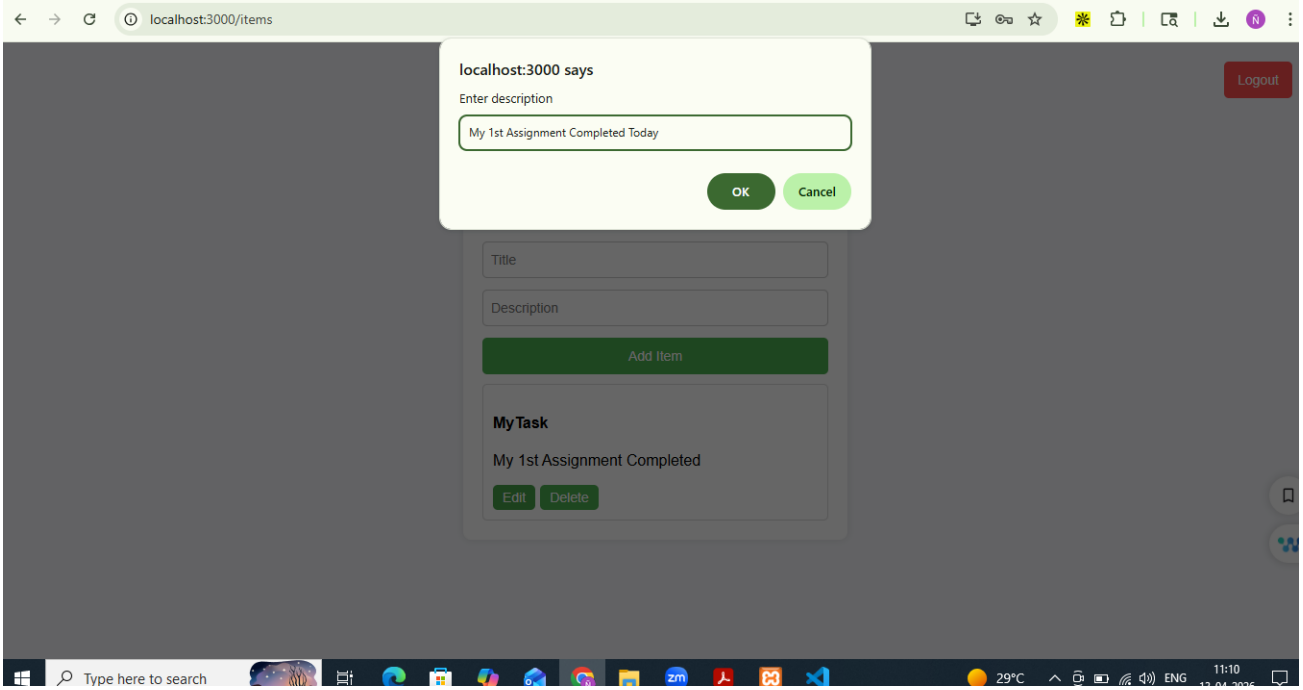

Fig 1.8 Editing Description

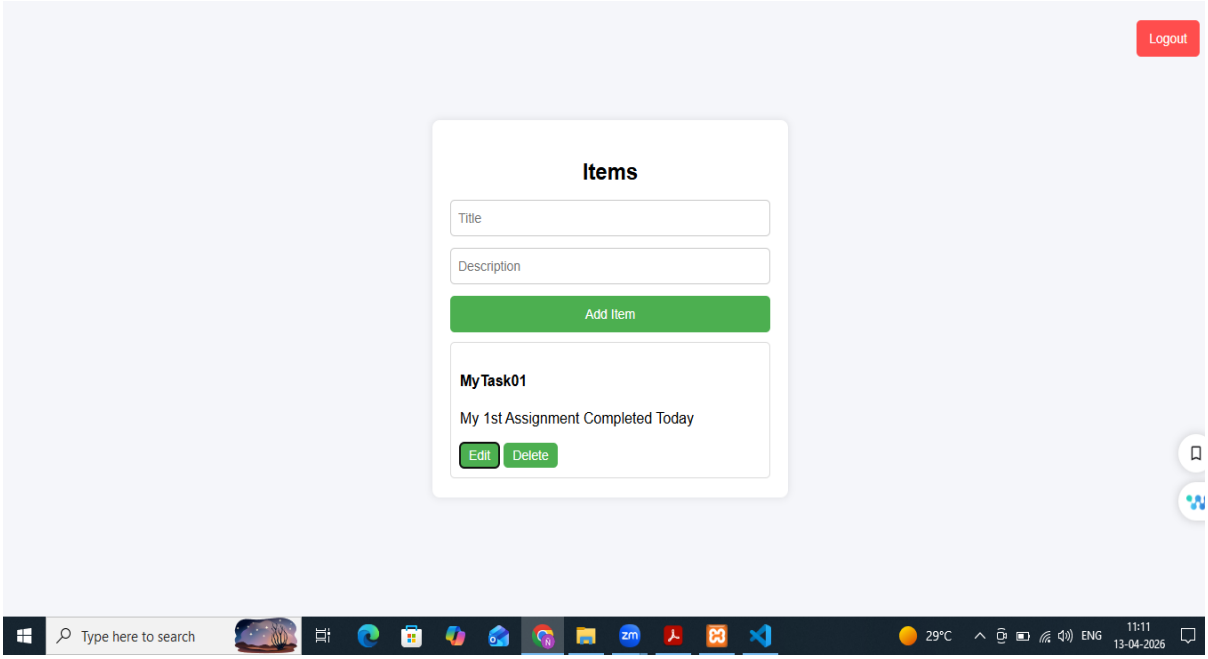

Fig 1.9 Edited Data

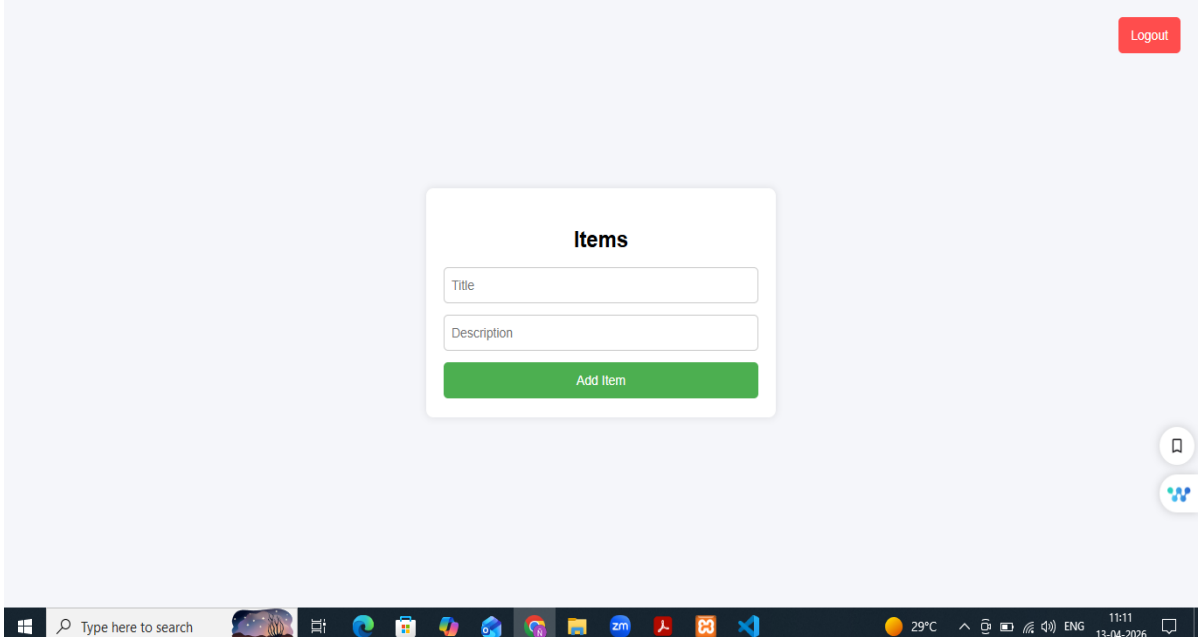

Fig 1.10 After deleting items

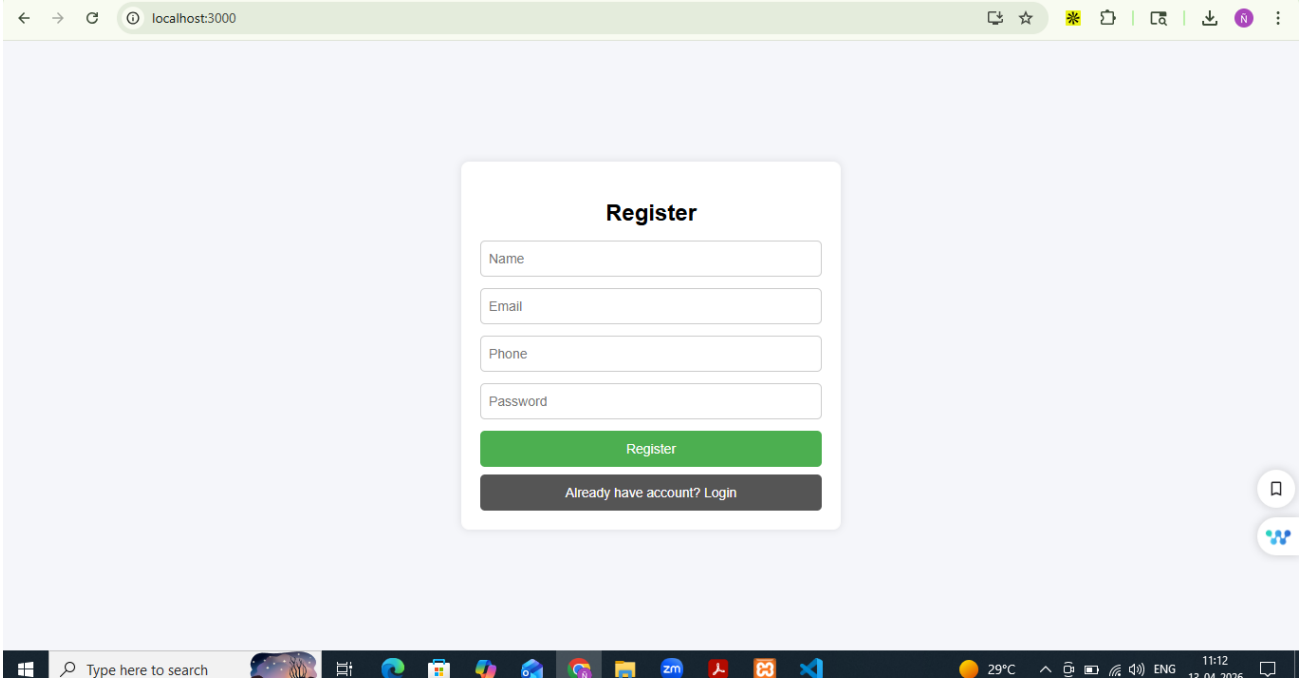

Fig 1.11 After logging out(Redirects to Regidter page)

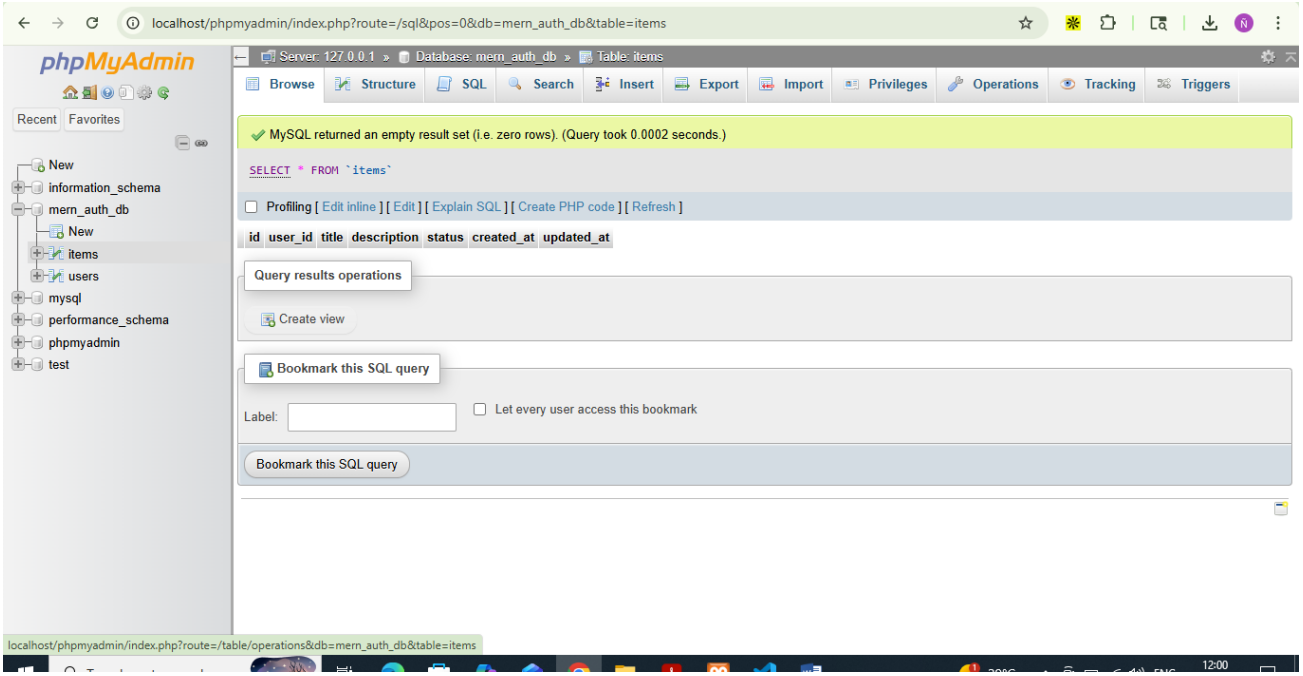

Fig 1.2 MySQL database showing tables with data(items)

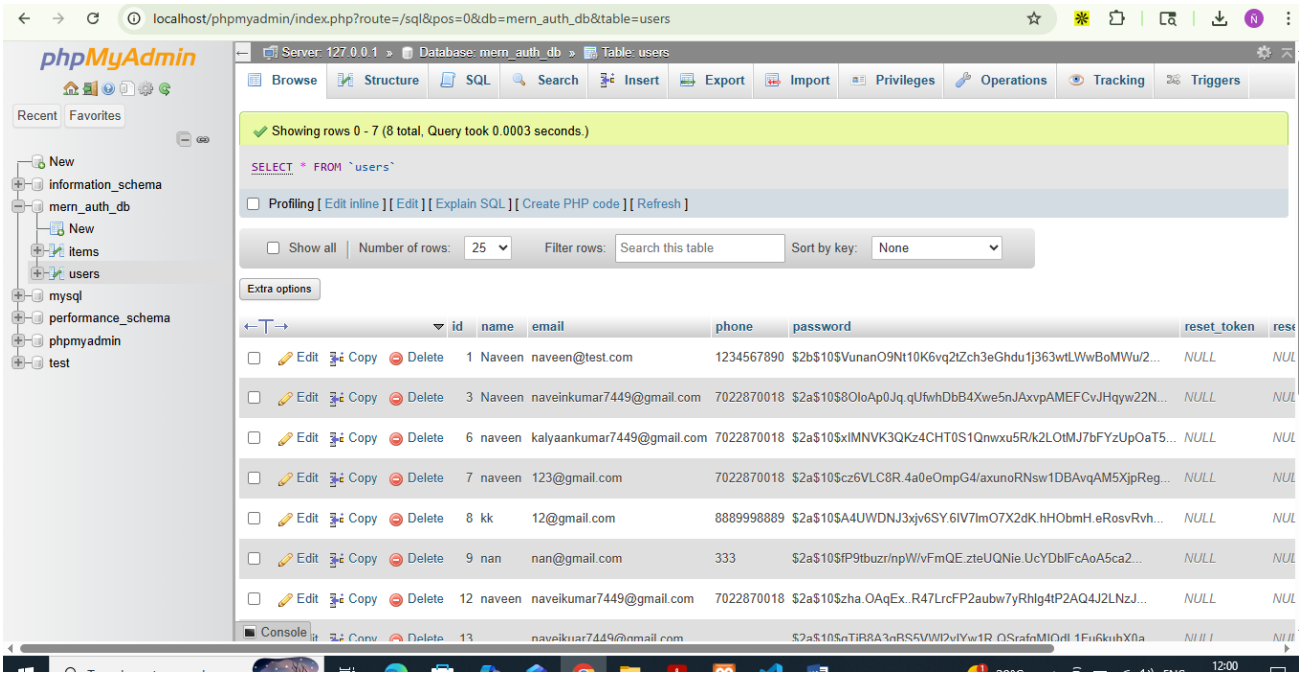

Fig 1.13 MySQL database showing tables with data(users)